



RÉGION AUVERGNE-RHÔNE-ALPES

Trouve ton artisan

Plateforme de mise en relation artisans / particuliers

Renaud VAILLANT

Parcours Développeur Web

Projet Full Stack — React · Node.js · MySQL

Juillet 2026

Dossier de projet — Document confidentiel

Sommaire

1. Contexte du projet	p. 3
1.1 Présentation de l'entreprise	
1.2 Expression des besoins	
1.3 Contraintes techniques	
1.4 Livrables attendus	
2. Maquettes Figma	p. 5
2.1 Maquette mobile	
2.2 Maquette tablette	
2.3 Maquette desktop	
2.4 Lien vers les maquettes complètes	
3. Base de données	p. 9
3.1 Modèle Conceptuel de Données (MCD)	
3.2 Modèle Logique de Données (MLD)	
3.3 Modèle Physique de Données (MPD)	
4. Sécurité	p. 12
4.1 Architecture de sécurité	
4.2 Mesures mises en place	
4.3 Variables d'environnement	
5. Veille sécurité	p. 15
5.1 Méthodologie de veille	
5.2 Vulnérabilités identifiées (OWASP Top 10)	
5.3 Failles corrigées dans le projet	
6. Liens du projet	p. 18

1. Contexte du projet

1.1 Présentation de l'entreprise

La **Région Auvergne-Rhône-Alpes** est une collectivité territoriale couvrant 12 départements dans le quart sud-est de la France. Elle dispose de bureaux à Lyon et à Clermont-Ferrand. Le projet est mené en relation avec l'antenne de Lyon, située au :

101 cours Charlemagne — CS 20033
69269 LYON CEDEX 02 — France
+33 (0)4 26 73 40 00
www.auvergnerhonealpes.fr

L'artisanat occupe une place majeure dans l'économie régionale : près d'un tiers des entreprises de la région sont des entreprises artisanales, soit **221 000 entreprises** recensées en 2021. La région Auvergne-Rhône-Alpes est ainsi l'une des régions les plus artisanales de France, et souhaite entretenir cet engouement par la création d'une plateforme numérique dédiée.

1.2 Expression des besoins

La Région Auvergne-Rhône-Alpes mandate la création d'une **plateforme web dédiée aux artisans de la région**, permettant aux particuliers de les trouver facilement et de leur adresser des demandes de renseignements, prestations ou tarifs via un formulaire de contact.

Le site devra :

Besoin	Description
Recherche d'artisans	Par catégorie (Alimentation, Bâtiment, Fabrication, Services) et par nom/métier/ville
Fiche artisan	Nom, photo/icône, note, spécialité, localisation, à propos, site web, formulaire de contact
Contact	Formulaire (nom, email, objet, message) envoyant un e-mail à l'artisan concerné
Artisans en vedette	3 artisans mis en avant sur la page d'accueil
Accessibilité	Conforme WCAG 2.1 — accessible à tous (jeunes, seniors, personnes en situation de handicap)
Responsive / Mobile First	Adapté à tous les formats d'écran, conçu en priorité pour mobile
Cohérence graphique	Respect de l'identité visuelle de la Région Auvergne-Rhône-Alpes

1.3 Contraintes techniques

Domaine	Technologie imposée
Maquettage	Figma

Domaine	Technologie imposée
Frontend	ReactJS, Bootstrap 5, Sass/SCSS
Backend (API REST)	Node.js, Express, Sequelize (ORM)
Base de données	MySQL 8 ou MariaDB 10.6+
Versionning	Git & GitHub
Éditeur	Visual Studio Code
Validation	Conformité W3C (HTML & CSS)
Hébergement	Site accessible en ligne
Sécurité	Bonnes pratiques (CORS, injection, XSS, rate limiting...)

1.4 Livrables attendus

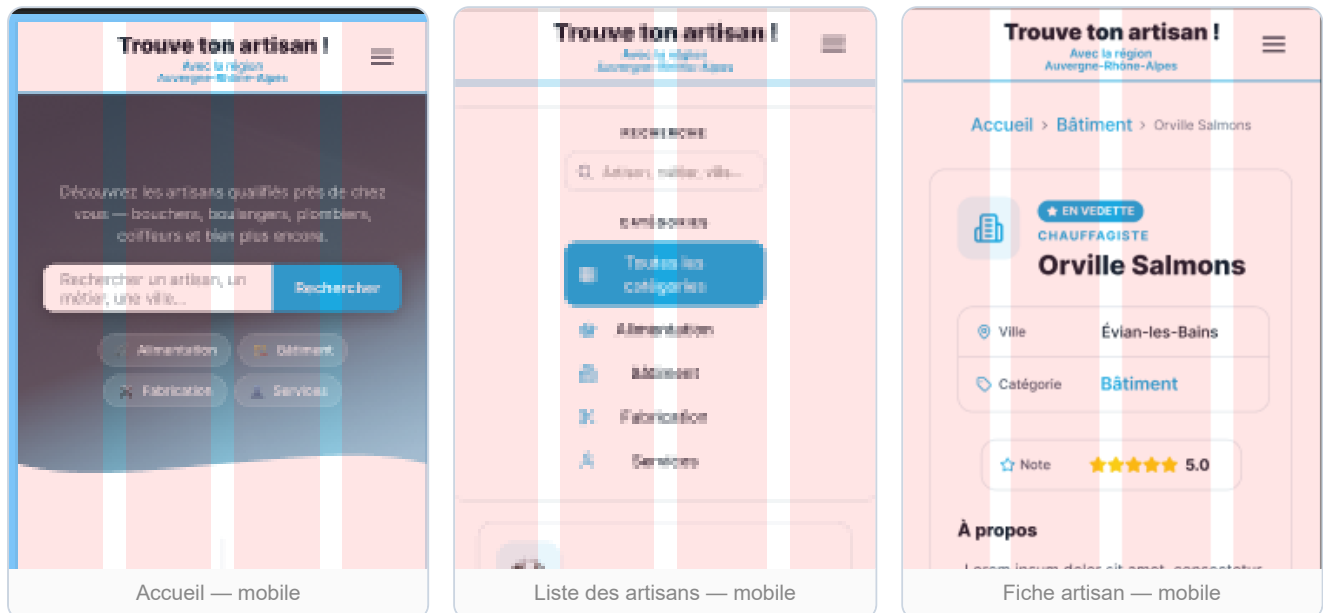
Livable	État
Maquettes Figma (mobile, tablette, desktop)	✓ Réalisé
Application web complète (frontend + API)	✓ Réalisé
Scripts SQL de création et d'alimentation	✓ Réalisé
Dépôt GitHub public	✓ Réalisé
Site hébergé en ligne	✓ Réalisé
Dossier PDF (ce document)	✓ Réalisé

2. Maquettes Figma

Lien vers les maquettes complètes :

figma.com/design/ReZaA7msVdrJ8du6tEPNcQ/Trouve_Ton_Artisan

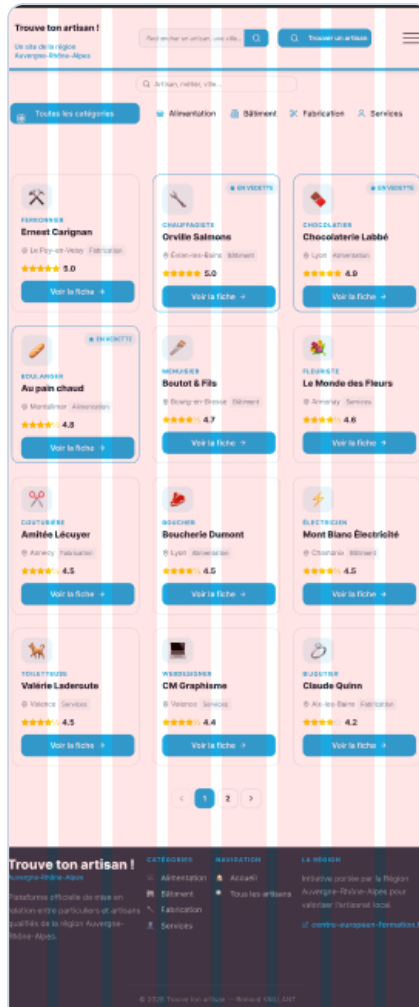
2.1 Maquette mobile (375px)



2.2 Maquette tablette (768px)



Accueil — tablette



Liste des artisans — tablette



Fiche artisan — tablette

2.3 Maquette desktop (1280px)



2.4 Identité graphique appliquée

Charte graphique imposée par le cahier des charges — les 6 couleurs ci-dessous sont reprises telles quelles dans les variables Sass du projet (`frontend/src/styles/_variables.scss`).

Élément	Valeur	Usage
Police principale	Graphik (Regular 400 · Medium 500 · Bold 700)	Toutes les pages
Bleu principal	#0074C7	Boutons, liens, accents
Bleu foncé	#00497C	Navigation, footer, hover
Gris anthracite	#384050	Texte principal
Rouge	#CD2C2E	Alertes, erreurs
Vert	#82B864	Succès, confirmation
Fond clair	#F1F8FC	Fonds de section
Étoiles (notation)	#F59E0B — Ambre	Hors charte — couleur neutre conventionnelle pour la notation
Logo	Fourni par la Région Auvergne-Rhône-Alpes	Header, footer

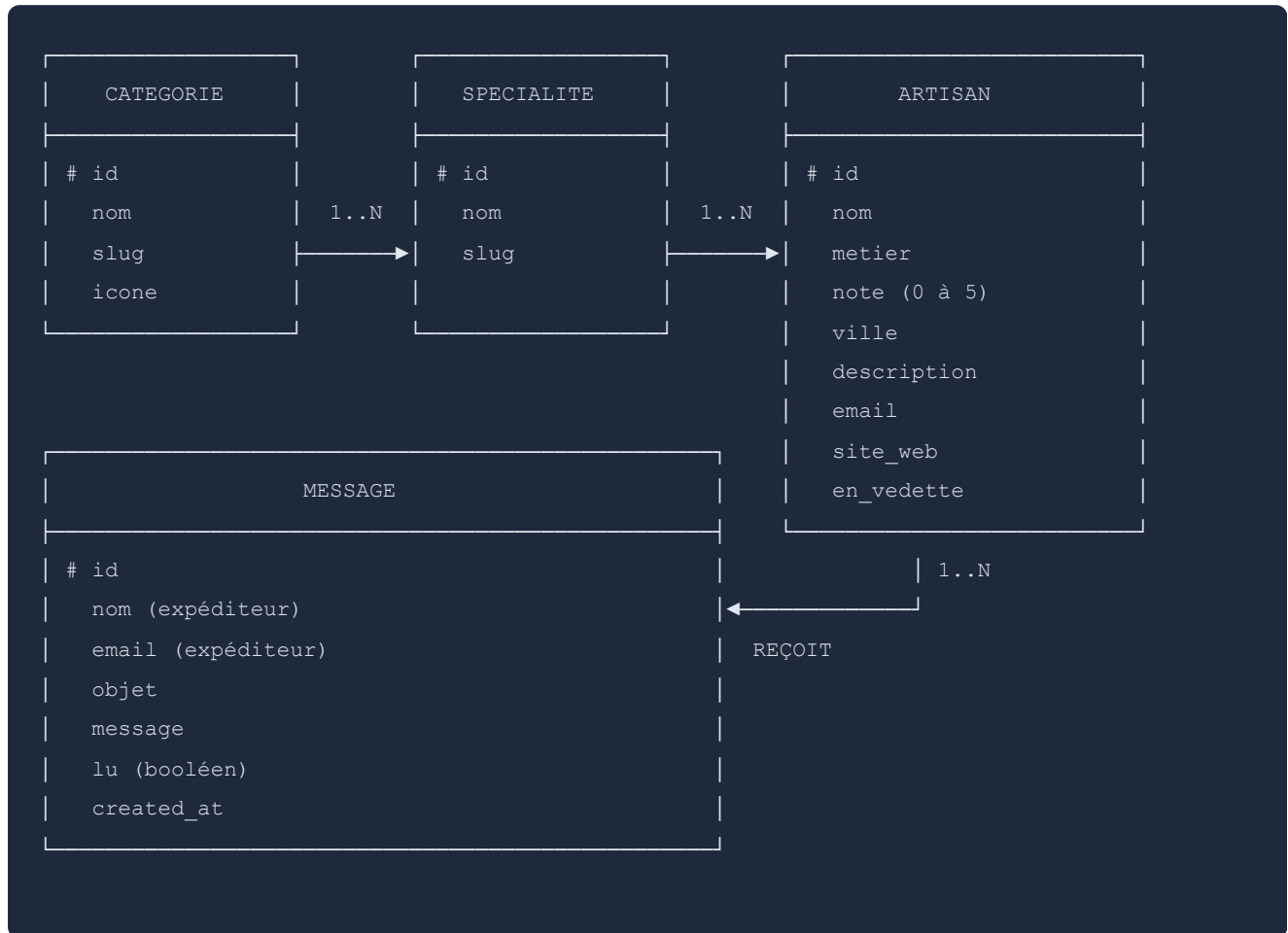
Élément	Valeur	Usage
Favicon	Fourni par la Région Auvergne-Rhône-Alpes	Onglet navigateur

3. Base de données

3.1 Modèle Conceptuel de Données (MCD)

Le cahier des charges précise les règles de gestion suivantes :

- Un artisan appartient à **une seule spécialité**
- Une spécialité est rattachée à **une seule catégorie**
- Un artisan peut recevoir **plusieurs messages** de contact



3.2 Modèle Logique de Données (MLD)

Règles de passage MCD → MLD : chaque association (1,N) génère une clé étrangère dans la table du côté N.

categories		
Champ	Type	Contrainte
PK id	INT UNSIGNED	AUTO_INCREMENT
nom	VARCHAR(100)	NOT NULL, UNIQUE
slug	VARCHAR(100)	NOT NULL, UNIQUE
icone	VARCHAR(60)	NULL

specialites		
Champ	Type	Contrainte
PK id	INT UNSIGNED	AUTO_INCREMENT
nom	VARCHAR(100)	NOT NULL
slug	VARCHAR(100)	NOT NULL, UNIQUE
FK categorie_id	INT UNSIGNED	→ categories(id)

artisans		
Champ	Type	Contrainte
PK id	INT UNSIGNED	AUTO_INCREMENT
nom	VARCHAR(150)	NOT NULL
metier	VARCHAR(100)	NOT NULL
note	DECIMAL(2,1)	NOT NULL, CHECK (0 ≤ note ≤ 5)
ville	VARCHAR(100)	NOT NULL
description	TEXT	NULL
email	VARCHAR(150)	NOT NULL
site_web	VARCHAR(255)	NULL
en_vedette	TINYINT(1)	DEFAULT 0
FK categorie_id	INT UNSIGNED	NOT NULL → categories(id)
FK specialite_id	INT UNSIGNED	NULL → specialites(id)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP

messages			admins		
Champ	Type	Contrainte	Champ	Type	Contrainte
PK id	INT UNSIGNED	AUTO_INCREMENT	PK id	INT UNSIGNED	AUTO_INCREMENT
nom	VARCHAR(100)	NOT NULL	email	VARCHAR(150)	NOT NULL, UNIQUE
email	VARCHAR(150)	NOT NULL	password	VARCHAR(255)	NOT NULL (bcrypt)
objet	VARCHAR(200)	NOT NULL	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
message	TEXT	NOT NULL			
artisan_nom	VARCHAR(150)	NULL			
lu	TINYINT(1)	DEFAULT 0			
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP			

3.3 Jeu d'essai

Le jeu d'essai fourni par la Région comprend **17 artisans** répartis en 4 catégories et 15 spécialités :

Catégorie	Artisans	En vedette
Alimentation	Boucherie Dumont, Au pain chaud, Chocolaterie Labbé, Traiteur Truchon	Au pain chaud, Chocolaterie Labbé
Bâtiment	Orville Salmons, Mont Blanc Électricité, Boutot & Fils, Vallis Bellemare	Orville Salmons
Fabrication	Claude Quinn, Amitée Lécuyer, Ernest Carignan	—
Services	Royden Charbonneau, Leala Dennis, C'est Sup'Hair, Le Monde des Fleurs, Valérie Laderoute, CM Graphisme	—

4. Sécurité

4.1 Architecture de sécurité

L'application repose sur une **séparation stricte entre le frontend et le backend** :

- Le **frontend React** est une application statique : il ne contient aucun secret, aucune logique métier sensible, aucun accès direct à la base de données.
- Le **backend Node.js/Express** est la seule couche qui interroge MySQL. Toutes les opérations de données passent exclusivement par son API REST.
- La **base de données MySQL** n'est jamais exposée publiquement.

4.2 Mesures de sécurité mises en place

Clé API — Authentification des requêtes publiques

Chaque requête vers l'API doit inclure l'en-tête `X-API-Key`. Un middleware custom vérifie sa validité via une comparaison **timing-safe** (`crypto.timingSafeEqual`) pour éviter les attaques temporelles.

Aspect	Détail
Mise en œuvre	<code>backend/middleware/apiKey.js</code>
Intérêt	Empêche tout accès à l'API depuis une source non autorisée
En cas d'échec	Retourne <code>401 Unauthorized</code>

JWT — Authentification de l'espace administration

L'accès aux routes `/api/admin/*` est protégé par un token JWT signé avec une clé secrète.

Aspect	Détail
Mise en œuvre	<code>backend/middleware/auth.js</code> + <code>jsonwebtoken</code>
Intérêt	Sessions sécurisées sans état côté serveur (stateless)
Durée de vie	8 heures — renouvellement requis après expiration

En-têtes HTTP sécurisés — Helmet

En-tête HTTP	Rôle
<code>Content-Security-Policy</code>	Restreint les sources de scripts, styles et médias autorisés
<code>X-Frame-Options: DENY</code>	Protège contre le clickjacking (intégration en iframe)
<code>X-Content-Type-Options: nosniff</code>	Empêche le navigateur de deviner le type MIME
<code>Strict-Transport-Security</code>	Force HTTPS en production
<code>Referrer-Policy</code>	Contrôle les informations envoyées au serveur référent

Limitation de débit — Rate Limiting

Portée	Limite	Intérêt
API globale (dev)	2 000 requêtes / 15 min	Développement sans blocage
API globale (prod)	100 requêtes / 15 min	Protection contre le scraping et les attaques par force brute
Formulaire contact	5 messages / heure	Prévention du spam

Protection XSS — xss-clean

Toutes les données entrantes (body, query, params) sont nettoyées automatiquement pour supprimer les balises HTML et scripts malveillants avant tout traitement.

Aspect	Détail
Mise en œuvre	Middleware <code>xss-clean</code> appliqué globalement
Intérêt	Neutralise les attaques XSS (Cross-Site Scripting) avant qu'elles atteignent la logique applicative

Protection HPP — HTTP Parameter Pollution

Le middleware `hpp` empêche les attaques par duplication de paramètres dans les query strings (ex : `?categorie=foo&categorie=bar`).

CORS — Cross-Origin Resource Sharing

L'API n'accepte les requêtes que depuis les origines explicitement autorisées. Toute requête d'une origine non listée est refusée par le navigateur avant même d'atteindre le serveur.

Protection contre les injections SQL — Sequelize ORM

L'ORM Sequelize utilise des **requêtes paramétrées** (prepared statements) pour toutes les interactions avec MySQL. Aucune interpolation directe de chaîne SQL n'est présente dans le code, ce qui rend les injections SQL impossibles.

Validation des entrées — express-validator

Tous les champs du formulaire de contact sont validés côté serveur avant traitement :

Champ	Validation appliquée
<code>nom</code>	Non vide, entre 2 et 100 caractères, caractères spéciaux échappés
<code>email</code>	Format email valide, normalisé
<code>objet</code>	Non vide, entre 2 et 200 caractères, échappé
<code>message</code>	Non vide, entre 10 et 1 000 caractères, échappé
<code>artisan_id</code>	Entier positif valide

Masquage des données sensibles

Le champ `email` des artisans est exclu de toutes les réponses API via Sequelize (`attributes: { exclude: ['email'] }`). L'email n'est utilisé qu'en interne côté serveur pour l'envoi du formulaire de contact.

Hachage des mots de passe — bcryptjs

Le mot de passe de l'administrateur est haché avec `bcryptjs` (coût 10) avant stockage. Le mot de passe en clair n'est jamais persisté en base de données.

4.3 Variables d'environnement

Tous les secrets sont stockés dans un fichier `.env` non versionné (présent dans `.gitignore`). Un fichier `.env.example` sans valeurs sensibles est fourni dans le dépôt.

Variable	Rôle
<code>DB_HOST / DB_USER / DB_PASSWORD</code>	Connexion à la base de données MySQL
<code>API_KEY</code>	Clé d'authentification des requêtes publiques (≥ 32 caractères en prod.)
<code>JWT_SECRET</code>	Clé de signature des tokens administrateur
<code>MAIL_*</code>	Configuration SMTP pour l'envoi d'e-mails (optionnel)

5. Veille sécurité

5.1 Méthodologie de veille

Tout au long du projet, une veille active sur les vulnérabilités web a été menée à partir de plusieurs sources de référence :

Source	Type d'information
OWASP Top 10 — owasp.org	Référentiel mondial des 10 vulnérabilités web les plus critiques
CVE / NVD — nvd.nist.gov	Base de données des vulnérabilités connues (Common Vulnerabilities and Exposures)
npm audit	Vérification automatique des dépendances Node.js pour détecter des packages vulnérables
MDN Web Docs — developer.mozilla.org	Documentation de référence sur la sécurité des navigateurs (CSP, CORS, cookies)

5.2 Vulnérabilités OWASP Top 10 — Analyse et réponse

#	Vulnérabilité OWASP	Risque	Mesure appliquée dans ce projet
A01	Broken Access Control	Critique	Routes admin protégées par JWT. Clé API sur routes publiques. Middleware de vérification systématique.
A02	Cryptographic Failures	Critique	Mots de passe hachés via bcryptjs (coût 10). Secrets dans <code>.env</code> non versionné. HTTPS en production.
A03	Injection (SQL, NoSQL, OS)	Critique	ORM Sequelize avec requêtes paramétrées. Aucune concaténation de chaîne SQL dans le code.
A04	Insecure Design	Important	Architecture frontend/backend séparée. Données sensibles (emails artisans) non exposées via l'API publique.
A05	Security Misconfiguration	Important	Helmet configure automatiquement les en-têtes HTTP. CORS restrictif. NODE_ENV différencié dev/prod.
A06	Vulnerable Components	Important	<code>npm audit</code> exécuté régulièrement. Dépendances maintenues à jour. Versions verrouillées dans <code>package-lock.json</code> .
A07	Authentication Failures	Critique	Rate limiting (5 tentatives/heure sur le contact). JWT à durée limitée. Comparaison timing-safe pour la clé API.
A08	Data Integrity Failures	Important	Validation complète côté serveur avec express-validator (typage, longueur, format). Double validation frontend + backend.

#	Vulnérabilité OWASP	Risque	Mesure appliquée dans ce projet
A09	Logging & Monitoring Failures	Important	Morgan pour le logging des requêtes HTTP. Gestionnaire d'erreurs global. Logs différenciés dev/prod.
A10	Server-Side Request Forgery	Faible	Aucune requête serveur vers des URL externes non contrôlées. Le backend n'effectue pas de fetch vers des ressources distantes inconnues.

5.3 Failles identifiées et corrigées durant le projet

Faille 1 — Email artisan exposé dans l'API publique

Aspect	Détail
Description	Les adresses e-mail des artisans étaient retournées dans les réponses JSON de l'API publique (liste et fiche artisan).
Risque	Collecte automatisée d'adresses e-mail (harvesting) pouvant mener à du spam ou du phishing ciblé.
Correction	Exclusion du champ <code>email</code> via Sequelize : <code>attributes: { exclude: ['email'] }</code> sur toutes les routes publiques. L'email est utilisé uniquement côté serveur pour l'envoi du formulaire de contact.

Faille 2 — Absence de validation côté serveur du formulaire de contact

Aspect	Détail
Description	Le formulaire de contact n'effectuait aucune validation des données côté serveur. Un attaquant pouvait soumettre des données malformées, des scripts ou des chaînes très longues.
Risque	Injections XSS via le message, pollution de la base de données, envoi de spam massif via l'API.
Correction	Implémentation d'express-validator avec trim, escape, longueur maximale, et format email. Rate limiting spécifique (5 messages/heure) pour prévenir le spam.

Faille 3 — Secrets versionnés dans le dépôt Git

Aspect	Détail
Description	Les fichiers <code>.env</code> contenant les mots de passe, clés JWT et clés API risquaient d'être inclus dans le dépôt Git.
Risque	Exposition publique de tous les secrets de l'application si le dépôt est public (GitHub).
Correction	Ajout de <code>.env</code> et <code>**/.env</code> dans <code>.gitignore</code> . Fourniture d'un <code>.env.example</code> avec des valeurs fictives pour guider l'installation.

Faible 4 — CORS ouvert à toutes les origines

Aspect	Détail
Description	Sans configuration CORS, Express accepte les requêtes depuis n'importe quelle origine, permettant à n'importe quel site d'interroger l'API.
Risque	Attaques Cross-Origin : un site malveillant peut exploiter la session d'un utilisateur authentifié pour envoyer des requêtes à l'API.
Correction	Configuration CORS stricte avec une liste blanche d'origines autorisées (<code>localhost:5173</code> en dev, domaine de production en prod). Toute autre origine est refusée.

Faible 5 — Dépendances tierces vulnérables (audit du 26/07/2026)

Aspect	Détail
Description	<code>npm audit</code> a révélé 8 vulnérabilités dans les dépendances backend : DoS sur <code>body-parser</code> , <code>brace-expansion</code> , <code>qs</code> ; falsification de logs sur <code>morgan</code> ; et 7 CVE critiques sur <code>nodemailer</code> (injection SMTP, injection CRLF, contournement de la protection anti-SSRF).
Risque	Déni de service applicatif, logs corrompus, et — pour <code>nodemailer</code> — envoi de mails vers des domaines détournés ou exécution de commandes SMTP arbitraires via le formulaire de contact.
Correction	<code>npm audit fix</code> pour les 4 premières (mise à jour non-cassante). Mise à jour manuelle de <code>nodemailer</code> (6.10.1 → 9.0.3), testée sans régression sur l'envoi de messages. La dépendance <code>sqlite3</code> , inutilisée et incohérente avec l'imposition de MySQL par le cahier des charges, a été supprimée.
Vulnérabilité résiduelle assumée	Une faille modérée sur <code>uuid</code> (dépendance transitive de Sequelize) n'a pas été corrigée : la résolution automatique proposait de rétrograder Sequelize en version 3 (obsolète), ce qui aurait cassé l'ensemble de l'ORM. La faille ne s'exploite que si un buffer est fourni explicitement à une fonction <code>uuid</code> , ce que le code ne fait jamais — risque jugé nul, à réévaluer à la prochaine mise à jour majeure de Sequelize.

6. Liens du projet

Ressource	Lien
Dépôt GitHub	github.com/thrax28700/Trouve_Ton_Artisan
Site en ligne	trouvetonartisan-production-9e7c.up.railway.app
Maquettes Figma	figma.com/design/ReZaA7msVdrJ8du6tEPNcQ/Trouve_Ton_Artisan
Validation W3C HTML	validator.w3.org (Nu Html Checker) — 0 erreur
Validation W3C CSS	jigsaw.w3.org — 0 erreur (validity: true)

Résultats détaillés de la validation W3C

Le HTML final (après rendu React) a été validé page par page via l'API du Nu Html Checker :

Page	Résultat
Accueil (/)	0 erreur
Liste des artisans (/artisans)	0 erreur
Fiche artisan (/artisans/:id)	0 erreur
Page 404	0 erreur
Page légale (mentions légales)	0 erreur

Deux erreurs réelles ont été détectées puis corrigées durant cette validation :

- **<main> imbriqué** — les pages Liste des artisans et 404 plaçaient un second `<main>` à l'intérieur du `<main>` global du layout (interdit par la spec HTML5, un seul `<main>` visible par document). Remplacé par `<div role="region">`.
- **aria-label invalide sur ** — le badge « En vedette » portait un `aria-label` redondant avec son texte visible, interdit sur un élément de rôle générique. Attribut supprimé (le texte visible suffit pour l'accessibilité).
- **Saut de niveau de titre** — sur la page 404, le `<h1>` était suivi directement d'un `<h3>` dans le pied de page (aucun `<h2>` intermédiaire). Les titres de colonnes du footer sont passés de `<h3>` à `<h2>`.

La validation CSS du bundle compilé (`frontend/dist/assets/*.css`) ne remonte **aucune erreur** ; les 262 avertissements sont uniquement des préfixes vendeurs (`-webkit-`, `-moz-`, `-o-`) injectés automatiquement par l'autoprefixer de Bootstrap/Vite et des avertissements attendus sur les variables CSS (`--*`), non évaluables statiquement par le validateur — aucun n'indique une erreur de code.

Contenu du dépôt GitHub

```

trouve-ton-artisan/
├── backend/
│   ├── controllers/      # Logique métier (artisans, contact, auth, admin)
│   ├── middleware/       # Authentification JWT, clé API, erreurs
│   ├── models/           # Modèles Sequelize (Artisan, Categorie, Specialite,
Message, Admin)
│   ├── routes/           # Routes Express (artisans, categories, contact, auth,
admin)
│   ├── .env.example      # Variables d'environnement (sans secrets)
│   ├── app.js            # Configuration Express et middlewares
│   ├── seed.js           # Script d'alimentation automatique
│   └── server.js         # Point d'entrée du serveur
├── database/
│   ├── create.sql        # Script de création des tables MySQL ✓
│   ├── insert.sql        # Script d'alimentation du jeu d'essai ✓
│   ├── MCD.md            # Modèle Conceptuel de Données
│   └── MLD.md            # Modèle Logique de Données
├── frontend/
│   └── src/
│       ├── components/   # Header, Footer, ArtisanCard, SearchBar, StarRating
│       ├── context/      # AuthContext (JWT côté client)
│       └── pages/        # Home, ArtisanList, ArtisanDetail, Admin/*, Legal/*,
NotFound
│       ├── services/     # api.js (public), adminApi.js (admin)
│       ├── styles/       # Variables SCSS, styles globaux
│       └── utils/        # slug.js (utilitaire partagé)
├── SECURITE.md           # Documentation sécurité détaillée
├── README.md             # Prérequis et instructions d'installation
└── start.ps1             # Script de démarrage Windows
    
```